

Visión Artificial

Trabajo Práctico N.º 4 · Informe de avance

Detección de personas y vehículos en imágenes aéreas para
plataformas de drones

Comparación YOLO11n / YOLO26n sobre VisDrone, reentrenamiento con las clases de la
misión y plan de benchmark en computadoras de a bordo (OBC) con y sin GPU

Integrantes	DNI
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]
[Apellido, Nombre]	[00.000.000]

Docentes: [completar]

Destinatarios: equipo del proyecto (coordinación, equipo Jetson, equipo Rasp-
berry Pi 5)

Resumen

El objetivo del TP4 es detectar personas y vehículos en imágenes captadas por drones, con un detector lo bastante liviano para ejecutarse a bordo de la aeronave. Sobre VisDrone2019-DET se ajustaron dos detectores de tamaño *nano* preentrenados en COCO, YOLO11n y YOLO26n, con un protocolo idéntico (50 épocas, entrada de 640 px, lote 4 y semilla 42). En validación ambos resultaron prácticamente equivalentes: mAP50–95 de 16,03 % y 15,85 %.

Para acercar el trabajo a la misión, las diez clases se agruparon en persona y vehículo, y se fijó un objetivo operativo: detectar al menos una vez durante la pasada el 85 % de las personas, con el modelo a 5 FPS o más en la placa. Con la entrada de 640 px, el límite es el tamaño aparente: el 46 % de las personas mide menos de 8 px y el modelo encuentra solo el 27 % de ellas en cada cuadro. Sin reentrenar, procesar la imagen a 1280 px sube ese recall al 43 % y el mAP50 de la misión de 48,4 % a 61,4 %. Con esa evidencia, ambos modelos se reentrenaron con persona y vehículo desde las etiquetas y con entrada de 1280 px, y esas dos clases pasaron a usarse en todo el trabajo. YOLO26n reentrenado alcanza un mAP50 de la misión de 69,9 % y encuentra el 57 % de las personas en cada cuadro.

En videos de VisDrone-VID, usando solo secuencias que no comparten origen con el entrenamiento, a 5 FPS simulados y con confianza 0,25, las personas detectadas al menos una vez pasan de 74 % (línea base a 640 px) a 84 % (línea base a 1280 px) y a 89 % con el modelo reentrenado, por encima del objetivo, con una sola inferencia por cuadro y las mismas falsas alarmas. Los vehículos pequeños de un video 4K siguen lejos del objetivo. El siguiente paso es medir YOLO26n reentrenado en una NVIDIA Jetson (GPU integrada, TensorRT) y en una Raspberry Pi 5 (solo CPU, NCNN), con el paquete de benchmark ya regenerado. Este informe se genera a partir de la wiki del proyecto.

Palabras clave: detección de objetos, vehículos aéreos no tripulados, YOLO, VisDrone, computación embarcada, TensorRT, NCNN.

Síntesis para el equipo

Qué se hizo	Datos preparados y auditados; YOLO11n y YOLO26n entrenados y comparados; clases de la misión definidas; evaluación a 640, 960 y 1280 px y con mosaicos; detección por pasada medida en VisDrone-VID; ambos modelos reentrenados con persona y vehículo a 1280 px; paquete de benchmark OBC regenerado con YOLO26n reentrenado; wiki del proyecto como fuente del informe.
Qué se concluye	Reentrenado con persona y vehículo a 1280 px, YOLO26n detecta al menos una vez el 89 % de las personas por pasada a 5 FPS (86 % a 2 FPS) con una inferencia por cuadro, frente a 84 % de la línea base a 1280 px y 74 % a 640 px. El costo sigue siendo de unas 7 falsas personas por cuadro con confianza 0,25. Los vehículos pequeños en 4K no llegan (65 %).
Qué sigue	Medir YOLO26n reentrenado a 1280 px en Jetson (TensorRT) y Raspberry Pi 5 (NCNN); ampliar el video con secuencias sin fuga; evaluar test-dev con el modelo fijado.
Qué decidir	Modelo exacto de Jetson y responsables; escenario de la misión (altura, velocidad, FPS mínimo); umbral de confianza y uso de un tracker para confirmar objetos; YOLO11n como control; medición de energía.

Índice

Resumen	2
1. Introducción	5
1.1. Visión artificial embarcada en drones	5
1.2. Planteo del problema	5
1.3. Objetivos	5
2. Datos	6
2.1. VisDrone2019-DET	6
2.2. VisDrone2019-VID	6
3. Metodología de la comparación inicial	8
3.1. YOLO11n	8
3.2. YOLO26n	8
3.3. Protocolo de entrenamiento	8
3.4. Métricas de detección	9
4. Resultados de la comparación inicial	10
4.1. Métricas globales	10
4.2. Resultados por clase	10
4.3. Evolución durante el ajuste	11
4.4. Análisis cualitativo de errores	12
4.5. Discusión y elección del modelo para las placas	12
5. Clases de la misión y evaluación agrupada	13
5.1. Clases de la misión	13
5.2. Evaluación por clases de la misión	13
6. Del laboratorio al dron: requisitos y objetivo	15
6.1. Requisitos operativos	15
6.2. Objetivo de detección por pasada	16
6.3. Detección por objeto durante la pasada	17
7. Mejoras de la inferencia	18
7.1. Resolución de entrada y mosaicos	18
7.2. Evaluación de resolución y mosaicos	19
8. Detección por pasada en video	21
8.1. Método	21
8.2. Detección al menos una vez	21
8.3. Personas: umbral, confirmación y falsas alarmas	22
8.4. Combinaciones de mosaicos y resolución	22
8.5. Lectura	23
9. Reentrenamiento con las clases de la misión	24
9.1. Qué se cambió	24
9.2. Validación	24
9.3. Comparación con la línea base	24
9.4. Resolución de entrada con el modelo reentrenado	25
9.5. Umbral de confianza	25
9.6. Referencia en test-dev	26
9.7. Conclusiones	26
10. Benchmark en computadoras de a bordo	26
10.1. Plataformas y variantes	26
10.2. Benchmark en placas	26
10.3. Indicadores de rendimiento en placa	29

11. Plan de trabajo	30
11.1. Tareas y criterios de aceptación	30
11.2. Decisiones	32
12. Limitaciones y riesgos	33
13. Conclusiones	34
Referencias	34
A. Anexo: trazabilidad de artefactos	35

1. Introducción

1.1. Visión artificial embarcada en drones

Los vehículos aéreos no tripulados (VANT, o UAV por su sigla en inglés) se emplean en vigilancia, búsqueda y rescate, monitoreo de tránsito, inspección de infraestructura y agricultura. En muchas de estas misiones, la percepción visual debe resolverse *a bordo*. Transmitir el video a una estación terrena para procesarlo añade latencia, depende de la calidad del enlace, consume ancho de banda y deja al sistema sin percepción si el enlace se pierde.

En la arquitectura habitual, un controlador de vuelo se encarga de la estabilización y la navegación. Junto a él, una *computadora de a bordo* (OBC, *On-Board Computer* o *companion computer*) ejecuta las tareas de mayor carga, como la detección de objetos. Esa computadora está sujeta a las restricciones SWaP (*Size, Weight and Power*): cada gramo y cada vatio reducen la autonomía de vuelo, la disipación térmica es limitada y no se dispone de una GPU de escritorio.

En este trabajo se consideran dos familias representativas: placas con GPU integrada (NVIDIA Jetson) y placas solo con CPU (Raspberry Pi 5). Ver plataformas.

1.2. Planteo del problema

La detección en imágenes aéreas es sensiblemente más difícil que en los conjuntos de referencia tomados desde el suelo, como COCO [1]:

- **Objetos muy pequeños.** Vistos desde decenas de metros, una persona o una bicicleta ocupan pocos píxeles. Al reducir la imagen al tamaño de entrada de la red pueden quedar por debajo del límite que el modelo logra resolver.
- **Alta densidad.** VisDrone promedia unos 53 objetos anotados por imagen en entrenamiento y 71 en validación¹. Las escenas urbanas contienen cientos de instancias parcialmente ocluidas.
- **Clases visualmente próximas.** *Pedestrian* frente a *people*, *tricycle* frente a *awning-tricycle* y *car* frente a *van* se diferencian por detalles que se pierden al reducir la resolución.
- **Variabilidad de punto de vista.** Hay tomas cenitales y oblicuas, con distintas alturas, iluminaciones y escenarios.

A esas dificultades se agrega la restricción de cómputo: el detector debe funcionar en tiempo real en la computadora de a bordo. Por lo tanto, la pregunta del proyecto no se reduce a *qué modelo es más preciso*. Interesa *qué combinación de modelo, motor de inferencia y placa ofrece el mejor compromiso entre precisión, velocidad, memoria y comportamiento térmico dentro de las restricciones de un dron*, medido con una métrica que tenga sentido para la misión: detectar a cada objeto al menos una vez durante la pasada.

1.3. Objetivos

Objetivo general. Evaluar la viabilidad de un detector YOLO de tamaño *nano* para detectar personas y vehículos a bordo de un dron, con una métrica alineada a la misión.

Objetivos específicos. La tabla detalla los objetivos específicos e indica cuáles están cumplidos y cuáles pendientes.

¹Conteos de `data/audit.json`: 343 204 cajas en 6471 imágenes de entrenamiento y 38 759 en 548 de validación.

	Objetivo específico	Estado
O1	Preparar y auditar VisDrone2019-DET en formato YOLO, con trazabilidad (SHA-256).	Cumplido
O2	Ajustar YOLO11n y YOLO26n bajo un protocolo idéntico y compararlos en validación.	Cumplido
O3	Construir un paquete de benchmark reproducible y portable para placas OBC.	Cumplido
O4	Medir en Jetson y Raspberry Pi 5 el modelo original y su versión optimizada.	Pendiente
O5	Comparar las plataformas y discutir el compromiso precisión/velocidad/memoria.	Pendiente
O6	Evaluar test-dev completo con el modelo definitivo.	Pendiente
O7	Definir las clases de la misión y evaluar el modelo agrupado sin reentrenar.	Cumplido
O8	Medir la detección por objeto durante la pasada en video (VisDrone-VID).	Cumplido
O9	Evaluar mejoras de inferencia (960 y 1280 px, mosaicos) sin reentrenar.	Cumplido
O10	Reentrenar con las clases de la misión a mayor resolución.	Cumplido
O11	Medir en las placas el costo de la entrada de 1280 px.	Pendiente

Tabla 1. Objetivos específicos del TP4 y estado al 28 de septiembre de 2026.

2. Datos

2.1. VisDrone2019-DET

VisDrone [2] es un conjunto de referencia de imágenes captadas por cámaras montadas en drones, en entornos urbanos y suburbanos de catorce ciudades de China. La tarea de detección (DET) anota diez categorías: *pedestrian*, *people*, *bicycle*, *car*, *van*, *truck*, *tricycle*, *awning-tricycle*, *bus* y *motor*. Se utilizaron los archivos distribuidos por Ultralytics [3], que replican los particionados oficiales: 6471 imágenes de entrenamiento, 548 de validación y 1610 de test-dev.

Las imágenes tienen una resolución nativa mayor que la entrada de la red: en validación y test-dev predominan 1400×788 y 1360×765 píxeles, y hay tomas de hasta 1920×1080 . Las 548 imágenes de validación provienen de 76 secuencias, pero están separadas por cientos de cuadros: la misma persona casi nunca aparece dos veces, y las anotaciones no tienen identificador de objeto. Por eso este conjunto no permite medir la detección durante una pasada; para eso está VisDrone-VID.

La conversión al formato YOLO mapea las categorías originales 1–10 a los índices 0–9 y excluye las demás. Las regiones marcadas con *score* 0 (zonas ignoradas) no se usan como etiquetas. La auditoría encontró una única caja inválida, de ancho 4 y alto 0, que se excluyó y quedó registrada. Todos los archivos originales se conservan junto con su hash SHA-256.

Partición	Imágenes	Cajas convertidas	Cajas por imagen	Regiones score 0
Entrenamiento	6471	343 204	53,0	10 345
Validación	548	38 759	70,7	1410
Test-dev	1610	75 102	46,6	2445

Tabla 2. Conteos reales obtenidos por la auditoría de datos (data/audit.json, generado por `python -m tp4.cli prepare`).

Nota de uso. Los datos se utilizan con fines académicos, según las condiciones publicadas por los autores de VisDrone. No se redistribuyen imágenes ni pesos fuera del equipo.

2.2. VisDrone2019-VID

Las tareas de video de VisDrone (VID y MOT) anotan cada cuadro con el formato `frame_index`, `target_id`, `bbox_left`, `bbox_top`, `bbox_width`, `bbox_height`, `score`, `object_category`, `truncation`, `occlusion` [4]. El campo `target_id` identifica a cada objeto a lo largo del video: con

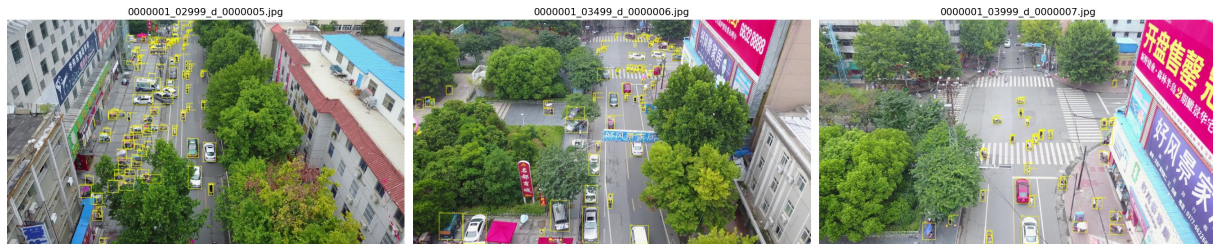


Figura 1. Revisión visual de cajas convertidas sobre tres imágenes de validación. La densidad y el tamaño reducido de peatones y motos ilustran la dificultad del dominio.

él se calcula la detección por objeto sin necesidad de un tracker. Las categorías son las mismas diez de DET, más regiones ignoradas (categoría 0, *score* 0) y *others* (11), así que el agrupamiento en clases de la misión se aplica sin cambios. El desafío oficial de MOT evalúa solo cinco clases; eso no limita el uso que hacemos acá.

2.2.1. Validación descargada

Se usó la validación de VID (1,6 GB, descarga manual desde la página oficial [2]), extraída en `data/raw/VisDrone2019-VID-val/` con su SHA-256 registrado².

Secuencia	Cuadros	Resolución	Personas	Vehículos	Video de origen en DET
uav0000086_00000_v	464	1344 × 756	79	3	validación
uav0000117_02622_v	349	2720 × 1530	105	40	validación
uav0000137_00458_v	233	2688 × 1512	81	99	entrenamiento
uav0000182_00000_v	363	1344 × 756	24	131	entrenamiento
uav0000268_05773_v	978	3840 × 2160	6	46	ninguno
uav0000305_00000_v	184	1904 × 1071	6	63	entrenamiento
uav0000339_00001_v	275	1904 × 1071	34	41	entrenamiento
Total	2846		335	423	

Tabla 3. Secuencias de validación de VisDrone-VID, objetos únicos por clase de la misión y relación con DET.

2.2.2. Verificaciones

- **Identificadores consistentes.** Solo uno de los 758 objetos cambia de clase de la misión a lo largo del video; se le asigna la más frecuente.
- **Fuga con DET confirmada.** Las imágenes de DET se extrajeron de los mismos videos: el prefijo 00000182_00000_d de DET corresponde a uav00000182_00000_v. Una comparación por huella perceptual (dHash) encontró cuadros idénticos. Cuatro secuencias tienen su video de origen en el **entrenamiento** de DET (entre 6 y 18 imágenes cada una), así que el modelo ya vio esas escenas. Por eso los resultados en video se informan aparte para las tres secuencias **limpias** (086, 117 y 268). Las secuencias 086 y 117 comparten origen con la validación de DET, que no se usó para entrenar.
- **Sin cuadros por segundo en los archivos.** Se asume el nominal de 30 FPS. Los objetos se desplazan unos pocos píxeles entre cuadros consecutivos, lo que es coherente con video continuo, pero el valor no está verificado. Por eso los submuestreos se informan también como "1 de cada k cuadros".

²`data/archives/VisDrone2019-VID-val.json`.

- **Resoluciones altas.** Tres secuencias superan los 2600 px de ancho, y una es 4K. A 640 px, un objeto de 4K se reduce 6 veces.

3. Metodología de la comparación inicial

3.1. YOLO11n

Generación consolidada de Ultralytics [5] dentro de la familia de detectores de una etapa YOLO [6]. Su posprocesamiento requiere supresión de no máximos (NMS). Se usó la variante más pequeña (*nano*), preentrenada en COCO. En la comparación resultó descriptivamente más favorable en mAP50–95 y en costo de entrenamiento, con una diferencia que no alcanza significancia (ver comparación).

3.2. YOLO26n

Generación más reciente de Ultralytics, también en variante *nano* y preentrenada en COCO. Según la documentación del fabricante [7], produce una salida *end-to-end* sin NMS, elimina el módulo DFL y está orientada explícitamente a dispositivos de borde. También declara mejoras en la función de pérdida pensadas para objetos pequeños; en este experimento esas mejoras no se tradujeron en una ganancia clara en las clases pequeñas.

Es el modelo de referencia del benchmark en placas: su checkpoint `best.pt` proviene de la ejecución 20260928T003017Z, reentrenada con persona y vehículo a 1280 px.

3.3. Protocolo de entrenamiento

En ambos modelos se reemplazó la cabeza de 80 clases de COCO por una propia y se ajustó la red completa (*transfer learning*). COCO aporta solo los pesos iniciales: sus clases no se comparan con las de VisDrone. Hubo dos etapas: la **línea base**, con las 10 clases de VisDrone a 640 px, y el **reentrenamiento**, con las clases de la misión a 1280 px.

Parámetro	Línea base	Reentrenamiento
Clases	10 de VisDrone	persona y vehículo
Épocas / resolución / lote	50 / 640 × 640 / 4	50 / 1280 × 1280 / 4
Semilla / <i>workers</i>	42 / 0	42 / 8
Selección de pesos	<code>best.pt</code> según validación	<code>best.pt</code> según validación
Ejecución YOLO11n	20260925T015117Z (3,27 h)	20260927T215941Z (2,51 h)
Ejecución YOLO26n	20260925T231501Z (4,41 h)	20260928T003017Z (2,93 h)

Tabla 4. Protocolo de cada etapa, con los mismos particionados, hardware (NVIDIA GeForce RTX 5070 Laptop de 8 GB, Windows 11) y software (Python 3.14.4, PyTorch 2.13.0+cu130, Ultralytics 8.4.162) (`config.yaml` y `run.json` de cada ejecución; entorno detallado en ESTADO.md).

El reentrenamiento tardó menos pese a procesar cuatro veces más píxeles. La causa probable es la carga de imágenes en CPU: en una prueba corta a 1280 px, pasar de 0 a 8 *workers* casi duplicó la velocidad (de 3,6 a 6,5 iteraciones por segundo).

Además se midieron:

- **Latencia:** predict completo en la RTX 5070, a la resolución de cada etapa, lote 1, con 5 calentamientos y 20 repeticiones sobre una misma imagen.
- **Auditoría visual de errores:** las 8 primeras imágenes de validación (871 objetos anotados), con confianza 0,25 e IoU 0,5, clasificando cada caja como verdadero positivo (TP), falso positivo (FP) o falso negativo (FN).

3.4. Métricas de detección

Se informan *Precision*, *Recall*, mAP50 (precisión media por clase con umbral de IoU de 0,5, promediada entre clases) y mAP50–95 (promedio sobre umbrales de IoU de 0,50 a 0,95 en pasos de 0,05). El mAP50–95 exige mayor exactitud de localización y fue el criterio principal para comparar modelos.

Todas las métricas provienen del evaluador de Ultralytics, **no** del evaluador oficial de VisDrone, que trata las regiones ignoradas de otra forma.

Para la misión interesan además métricas en un **punto de operación** (un umbral de confianza fijo): la precisión es la fracción de detecciones correctas y el recall, la fracción de objetos encontrados con la clase correcta e IoU de al menos 0,5. Y, sobre todo, la detección por objeto durante la pasada.

4. Resultados de la comparación inicial

4.1. Métricas globales

La tabla resume la evaluación de cada modelo sobre el conjunto de validación completo, junto con la latencia y el costo de entrenamiento³.

Métrica (validación)	YOLO11n	YOLO26n	Δ (26 – 11)
Precision	41,88 %	40,41 %	–1,47 p. p.
Recall	31,38 %	31,77 %	+0,39 p. p.
mAP50	28,41 %	28,45 %	+0,04 p. p.
mAP50–95	16,03 %	15,85 %	–0,18 p. p.
Latencia RTX 5070 (ms/imagen)	26,59	26,74	+0,15
Tiempo total de ajuste (h)	3,27	4,41	+34,9 %

Tabla 5. Evaluación de `best.pt` sobre validación (548 imágenes). Una ejecución por modelo; las diferencias no tienen significancia estadística establecida.

El rendimiento global es muy similar. YOLO11n muestra una ventaja descriptiva de 0,18 p. p. en mAP50–95 y YOLO26n una de 0,04 p. p. en mAP50. Con una sola semilla, ninguna de las dos diferencias permite afirmar que un modelo supere al otro.

La latencia en la laptop es prácticamente idéntica (unas 37 imágenes por segundo en ambos casos). Esta cifra *no* se puede extrapolar a una computadora de a bordo: allí cambian el hardware, el motor de inferencia y la precisión numérica.

YOLO26n requirió un 34,9 % más de tiempo de ajuste. Las vistas previas por época suman solo 10,3 s, por lo que no explican esa diferencia de unos 68 minutos. De todos modos, el costo de entrenamiento se paga una sola vez y fuera de línea, mientras que el costo de inferencia se paga en cada cuadro durante el vuelo.

4.2. Resultados por clase

Clase	AP50–95 YOLO11n	AP50–95 YOLO26n	Δ (p. p.)
pedestrian	12,28 %	12,93 %	+0,65
people	7,14 %	7,32 %	+0,17
bicycle	2,18 %	2,20 %	+0,02
car	47,14 %	46,23 %	–0,92
van	21,92 %	21,53 %	–0,39
truck	17,73 %	17,21 %	–0,52
tricycle	8,96 %	9,59 %	+0,63
awning-tricycle	5,44 %	5,23 %	–0,21
bus	25,28 %	24,09 %	–1,19
motor	12,25 %	12,22 %	–0,03

Tabla 6. AP50–95 por clase sobre validación.

³`analysis/comparacion_yolo11_yolo26.md`, con los `run.json` de ambas ejecuciones.

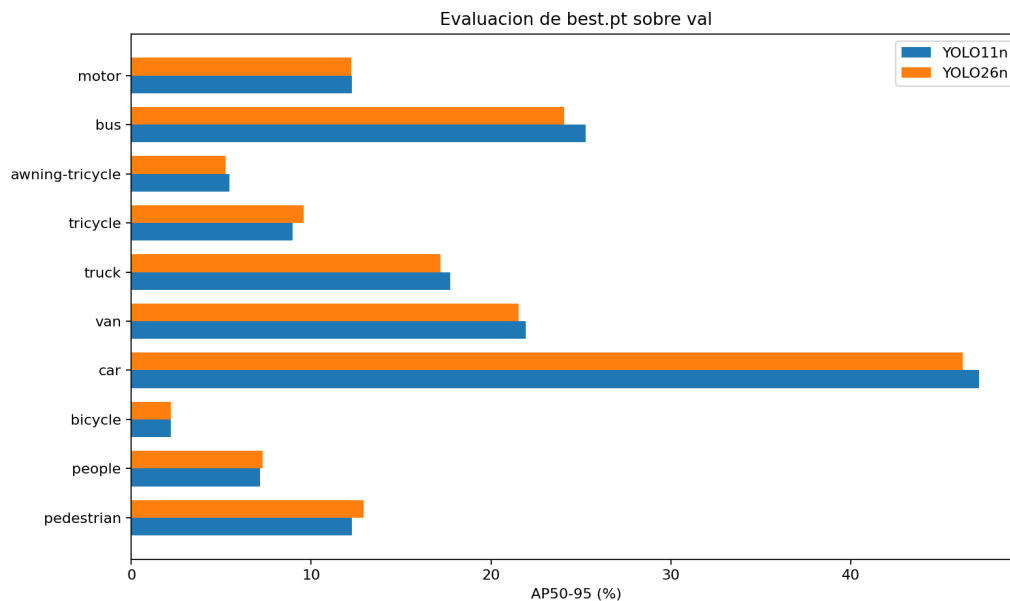


Figura 2. AP50–95 por clase para YOLO11n (azul) y YOLO26n (naranja).

Car es la clase más fuerte en ambos modelos (alrededor del 46–47 %): es un objeto relativamente grande y abundante. Las más débiles son *bicycle* (2,2 %), *awning-tricycle* (5,3 %) y *people* (7,2 %), todas pequeñas o visualmente ambiguas.

YOLO26n mejora levemente *pedestrian* y *tricycle*; YOLO11n obtiene más AP en *bus*, *car* y *truck*. Las mejoras para objetos pequeños que declara el fabricante de YOLO26 no se traducen, en este experimento, en una ganancia clara en las clases pequeñas: *bicycle* mejora 0,02 p. p. y *people*, 0,17 p. p.

4.3. Evolución durante el ajuste

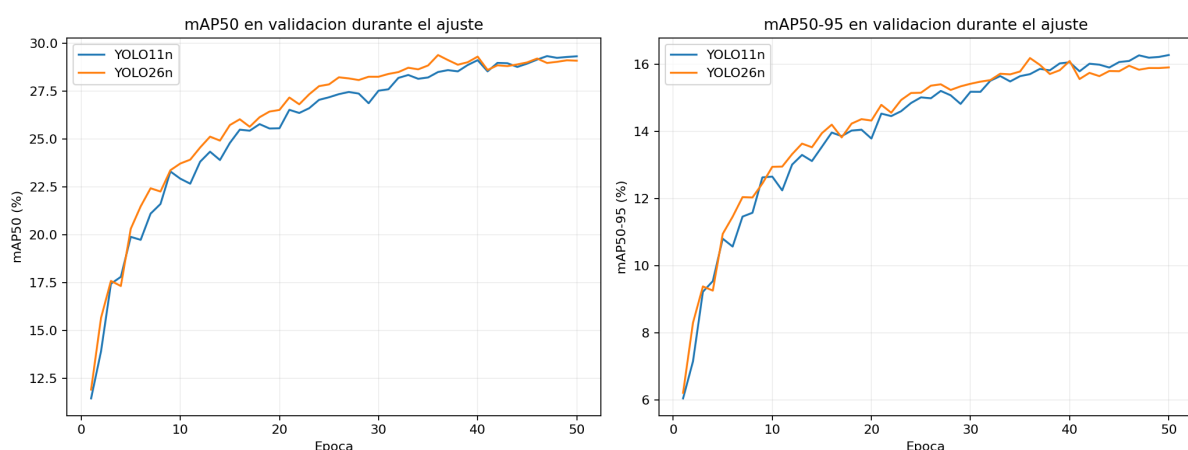


Figura 3. mAP50 y mAP50–95 en validación por época (validación interna de Ultralytics durante el ajuste).

Las curvas crecen con fuerza durante las primeras épocas y se aplanan hacia las épocas 35–40. El máximo de mAP50–95 de YOLO11n aparece en la época 50 (16,28 %). El de YOLO26n aparece en la época 36 (16,19 %) y baja a 15,91 % en la 50, una caída leve que no alcanza para afirmar sobreajuste. Las magnitudes de la pérdida no se comparan entre arquitecturas, porque sus componentes difieren (DFL frente a L1).

Las cifras de las curvas provienen de las validaciones internas; la tabla de métricas globales usa la evaluación independiente de `best.pt`. No son exactamente iguales y la causa no está aislada, por lo que no se mezclan ambos protocolos.

4.4. Análisis cualitativo de errores



Figura 4. Auditoría de errores de YOLO26n (confianza 0,25, IoU 0,5) sobre cuatro de las ocho imágenes de la muestra. Rojo: falsos negativos; cian: falsos positivos.

En la muestra de 871 objetos, YOLO11n obtuvo $TP = 263$, $FP = 192$ y $FN = 608$, y YOLO26n obtuvo $TP = 251$, $FP = 151$ y $FN = 620$. Es decir, YOLO26n comete menos falsas alarmas pero omite algunos objetos más.

La inspección muestra que predominan las omisiones de personas y vehículos de dos ruedas en escenas densas, junto con falsas detecciones en vehículos estacionados y vegetación. Se trata de una observación cualitativa sobre escenas relacionadas, no de una muestra representativa. Aun así, coincide con el patrón por clase: el cuello de botella son los objetos pequeños.

4.5. Discusión y elección del modelo para las placas

Los resultados muestran que **los dos modelos son equivalentes en precisión** para este problema, y que el techo actual lo imponen los objetos pequeños, no la arquitectura. En validación, YOLO11n es descriptivamente más favorable por mAP_{50-95} y costo de entrenamiento. Sin embargo, para un sistema embarcado importan otros factores:

1. **Posprocesamiento.** YOLO26n no requiere NMS. En escenas densas como las de VisDrone, el NMS procesa muchos candidatos y su costo recae en la CPU de la placa. Un modelo *end-to-end* simplifica la exportación y vuelve más predecible el tiempo por cuadro.
2. **Orientación a borde.** El fabricante diseñó YOLO26 para dispositivos de borde y declara mejoras de

velocidad en CPU [7]. Esa afirmación es exactamente lo que el benchmark en placas debe verificar.

3. **Costo de entrenamiento.** La diferencia de tiempo de ajuste no influye en la operación del dron.

Por eso se eligió **YOLO26n** como modelo de referencia para el benchmark en placa. Se trata de una decisión fundada en el diseño del modelo, no en una superioridad medida; las decisiones abiertas proponen incorporar YOLO11n como control para ponerla a prueba.

5. Clases de la misión y evaluación agrupada

5.1. Clases de la misión

La misión no necesita distinguir a un peatón de una persona sentada, ni un auto de una moto: necesita saber dónde hay personas y dónde hay vehículos. El equipo definió dos clases⁴, que son las mismas en todo el TP: entrenamiento, evaluación en la laptop y benchmark en las placas.

Clase de la misión	Clases VisDrone que agrupa
persona	<i>pedestrian, people</i>
vehículo	<i>car, van, truck, bus, tricycle, awning-tricycle, bicycle, motor</i>

Tabla 7. Agrupamiento de clases definido para la misión.

Criterios de la decisión:

- **Dos clases, no tres.** Se probó primero separar *dos ruedas* (bicicleta y moto). VisDrone anota a quien conduce una bicicleta o una moto como *people*, así que esa persona ya cuenta como persona. Además, en las secuencias de video limpias hay solo 10 bicicletas y motos. Sumarlas a vehículo no cambia los resultados de persona y sube la detección por pasada de vehículos a 1280 px de 61 % a 64 %⁵.
- **Agrupar, no borrar.** Si se quitaran las etiquetas de *van* o *truck* pero esos objetos siguieran en las imágenes, el modelo aprendería a tratarlos como fondo, y eso choca con aprender "vehículo".
- **No usar las clases de COCO.** COCO solo aporta los pesos iniciales. Usar directamente las clases de COCO sin reentrenar daría peor resultado, porque son fotos tomadas desde el suelo.
- **Sin efecto en velocidad.** Pasar de 10 a 2 clases solo cambia la última capa de clasificación; los FPS en la placa serían prácticamente iguales. La ganancia es de precisión y de claridad para la misión.
- **El objetivo se fija sobre personas.** Vehículo se informa, pero no se exige (ver el objetivo por pasada).

El agrupamiento se aplica **al preparar los datos**: cada etiqueta de VisDrone se convierte a su clase de la misión, y el modelo se entrena directamente con persona y vehículo. El modelo de línea base es anterior a esta decisión: se entrenó con las 10 clases y se evaluó agrupando sus predicciones después de la inferencia (ver la evaluación por clases de la misión). Los resultados del modelo entrenado con las dos clases están en el reentrenamiento a 1280 px.

5.2. Evaluación por clases de la misión

Esta evaluación corresponde al modelo de **línea base**: YOLO26n entrenado a 640 px con las diez clases de VisDrone. Los resultados del modelo reentrenado con persona y vehículo a 1280 px están en el

⁴MISSION_GROUPS en `tp4/data.py`, que usa la conversión de etiquetas.

⁵Evaluación en video a 5 FPS y confianza 0,25, secuencias limpias.

reentrenamiento.

5.2.1. Método

El detector conserva sus 10 clases. Se capturan las predicciones del propio validador de Ultralytics (entrada rectangular de 640 px, confianza 0,001, máximo 300 detecciones) y luego se reagrupan predicciones y etiquetas en las clases de la misión⁶. La variante de 10 clases, calculada con el mismo código, reproduce exactamente el mAP50 de 28,45 % y el mAP50–95 de 15,85 % registrados en `run.json`, lo que valida el procedimiento. La variante *con fusión* aplica además una supresión por grupo (IoU 0,7), para no contar dos veces un objeto que el modelo marcó a la vez como *car* y como *van*.

Se evaluó solo validación: test-dev queda reservado para la evaluación final.

5.2.2. Resultados

Variante	mAP50	mAP50–95
10 clases VisDrone (control)	28,5 %	15,9 %
Misión, agrupado directo	47,9 %	25,1 %
Misión, agrupado con fusión	48,8 %	25,3 %

Tabla 8. mAP en validación (548 imágenes) antes y después de agrupar las clases (`mission_eval/val.json` de la ejecución YOLO26n).

El salto parece grande, pero una tarea con menos clases es más fácil: estas cifras no se comparan con las de 10 clases, sino con futuros modelos evaluados con las mismas dos clases. Lo informativo es el detalle por clase en un punto de operación concreto.

Clase	Objetos	AP50	AP50–95	Precisión	Recall
persona	13 969	34,6 %	13,0 %	65,3 %	27,7 %
vehículo	24 790	63,0 %	37,5 %	77,3 %	54,2 %
todos	38 759			74,3 %	44,6 %

Tabla 9. Clases de la misión (agrupado con fusión). Precisión y recall con confianza 0,25 e IoU 0,5.

Agrupar corrige casi toda la confusión entre clases, que se concentraba en los vehículos: en la matriz de confusión de 10 clases, el 41 % de las *van* se predecía como *car* y el 17 % de los *truck* también⁷. En cambio, la confusión entre *pedestrian* y *people* era pequeña (2–6 %), porque la mayoría de esas personas directamente no se detecta. Vehículo incluye bicicletas y motos, que son pequeñas y bajan su recall frente a los vehículos de cuatro ruedas.

⁶`tp4/mission.py`; comando `python -m tp4.cli mission runs/full/20260925T231501Z`.

⁷`runs/full/20260925T231501Z/validation/confusion_matrix_normalized.png`, con el umbral por defecto.

5.2.3. Umbral de confianza

Confianza	persona P / R	vehículo P / R	todos P / R
0,05	25,4 % / 50,1 %	37,1 % / 71,6 %	32,8 % / 63,9 %
0,10	35,4 % / 45,0 %	49,9 % / 66,8 %	44,9 % / 59,0 %
0,15	45,8 % / 38,9 %	60,7 % / 62,3 %	56,0 % / 53,8 %
0,25	65,3 % / 27,7 %	77,3 % / 54,2 %	74,3 % / 44,6 %
0,40	86,5 % / 14,8 %	93,0 % / 41,5 %	91,9 % / 31,9 %
0,50	93,7 % / 8,9 %	97,4 % / 34,3 %	96,9 % / 25,2 %

Tabla 10. Precisión (P) y recall (R) por cuadro según el umbral de confianza, IoU 0,5.

Una precisión del 85 % se consigue subiendo la confianza a 0,4–0,5, a costa de que el recall de personas caiga al 15 % o menos. Un recall por cuadro del 85 % no se alcanza en ninguna clase con ningún umbral.

5.2.4. Recall según el tamaño del objeto

Clase	< 4 px	4–8 px	8–16 px	16–32 px	≥ 32 px
persona	0,5 % (1164)	11,8 % (5312)	38,2 % (5971)	61,9 % (1413)	78,9 % (109)
vehículo	2,3 % (1073)	23,3 % (4449)	48,4 % (9127)	73,9 % (7319)	90,0 % (2822)

Tabla 11. Recall con confianza 0,25 según el lado equivalente del objeto, raíz de ancho por alto, medido en la entrada de 640 px de la red. Entre paréntesis, la cantidad de objetos anotados.

El **46 % de las personas anotadas mide menos de 8 px** en la entrada de la red, y ahí el modelo prácticamente no detecta nada. El error dominante es no ver el objeto, no confundir la clase: por eso la siguiente palanca es la resolución de entrada, y no un reentrenamiento solo para agrupar.

5.2.5. Observación sobre el pipeline de inferencia

Un primer intento que evaluaba con `predict` en lugar del validador dio unos 3 puntos menos de mAP50 de 10 clases (25,6 % frente a 28,5 %) con los mismos pesos. La causa no está aislada. No afecta al benchmark, porque cada placa se compara contra el `.pt` medido con su mismo pipeline, pero implica que las cifras de las placas no se comparan directamente con las de validación.

6. Del laboratorio al dron: requisitos y objetivo

6.1. Requisitos operativos

Las métricas de los experimentos se obtuvieron sobre imágenes fijas y en una GPU de laptop. Para acercarnos al problema real conviene traducirlas a magnitudes de la misión: altura de vuelo, campo visual de la cámara, velocidad y cuadros por segundo.

6.1.1. Tamaño aparente de los objetos

Para una cámara con campo de visión horizontal θ , a una altura h y con apunte cenital, el ancho cubierto en el terreno y la resolución espacial (GSD, *Ground Sample Distance*) son:

$$W = 2 h \tan\left(\frac{\theta}{2}\right), \quad \text{GSD} = \frac{W}{N}, \quad n_{\text{px}} = \frac{L}{\text{GSD}} \quad (1)$$

donde N es el ancho en píxeles de la imagen que procesa la red y L el tamaño real del objeto.

Ejemplo ilustrativo (supuestos: $h = 50$ m, $\theta = 80^\circ$, toma cenital)

$W = 2 \cdot 50 \cdot \tan 40^\circ \approx 83,9$ m. Con la imagen nativa ($N = 1400$ px), el GSD es de unos 6,0 cm/px; con la entrada de la red ($N = 640$ px), de unos 13,1 cm/px. Una persona vista desde arriba ($L \approx 0,5$ m) pasa de ocupar unos **8 px** a unos **4 px**, y un auto ($L \approx 4,5$ m) pasa de unos 75 px a unos 34 px. Esto explica por qué *car* alcanza un AP alto mientras *people* y *bicycle* quedan muy por debajo: al reducir la imagen a 640 px, las personas quedan en el límite de lo que la red puede resolver.

La evaluación por clases de la misión confirma el ejemplo con datos: el recall de personas pasa de 11,8 % entre 4 y 8 px a 61,9 % entre 16 y 32 px. De aquí surgen dos palancas que deben evaluarse *en la placa*, porque ambas aumentan el costo de cómputo: aumentar la resolución de entrada y procesar la imagen por mosaicos, como propone SAHI [8] (ver mejoras de inferencia). Además, la altura de vuelo es una palanca de la misión: volar más bajo agranda a las personas en la imagen.

6.1.2. Frecuencia de procesamiento

Si el dron avanza a velocidad v y la placa procesa f cuadros por segundo, entre dos detecciones consecutivas el dron recorre

$$\Delta d = \frac{v}{f} \quad (2)$$

Por ejemplo, a $v = 10$ m/s, procesar 2 FPS implica 5 m entre detecciones, 5 FPS implica 2 m y 15 FPS implica unos 0,7 m. El umbral aceptable depende de la misión. Un conteo de personas en una zona casi estática tolera pocos FPS; el seguimiento de vehículos en movimiento o la evasión requieren muchos más y una latencia baja y estable. Por eso el benchmark informa, además de la media, los percentiles p50 y p95 de latencia: en control, el peor caso importa tanto como el promedio.

Los FPS también determinan cuántas oportunidades tiene el detector de ver a cada objeto durante la pasada, y por lo tanto el objetivo de detección por pasada.

6.2. Objetivo de detección por pasada

Objetivo del TP4: detectar **al menos una vez durante la pasada el 85 % de las personas**, con el modelo optimizado corriendo **a 5 FPS o más en la placa** y con entrada de **1280 px**.

- **Calidad:** 85 % de personas detectadas al menos una vez, con confianza 0,25, en las secuencias limpias de VisDrone-VID.
- **Velocidad:** 5 FPS sostenidos en la placa. A 10 m/s, el dron avanza 2 m entre detecciones (ver requisitos operativos).
- **Configuración por defecto:** imagen completa a 1280 px, una inferencia por cuadro. Es la que mejor equilibra calidad y costo; los mosaicos quedan como resultado de laboratorio.
- **Alcance:** el objetivo se fija sobre personas. Vehículo se informa, pero no se exige: los vehículos pequeños en 4K no alcanzan el 85 % con ninguna variante, ni siquiera reentrenando (65 % a 1280 px, 72 % con mosaicos nativos).

La calidad se mide en el laboratorio y la velocidad en la placa. El benchmark da los FPS que sostiene cada placa, y la evaluación en video da la detección por pasada a esos FPS.

Exigir el 85 % en cada cuadro no es realista con objetos de pocos píxeles: con el modelo de línea base, el recall por cuadro de personas era 27,7 % con confianza 0,25 a 640 px, y con el reentrenado a 1280 px es 57,4 % (ver evaluación por clases de la misión). En cambio, durante una pasada cada persona aparece en muchos cuadros y basta con verla una vez para registrarla. Este objetivo se mide con la detección por objeto sobre videos con identificador (VisDrone-VID).

"Al menos una vez" es un criterio generoso. Para que el número sea honesto se fijan estas reglas:

1. **Evaluar a los FPS reales de la placa.** Si el video tiene 30 cuadros por segundo y la Raspberry procesa 3, se evalúa uno de cada 10 cuadros. Así el benchmark en placas alimenta directamente esta métrica.
2. **Acompañarlo con falsas alarmas por minuto.** Si no, bajar la confianza a 0,05 infla el resultado gratis.
3. **Reportar también "detectado en al menos 3 cuadros"**, que es lo que necesita un tracker para confirmar un objeto, y el **tiempo hasta la primera detección**.
4. **Desglosar por tamaño máximo del objeto** durante la pasada, igual que en la evaluación por cuadro.

Estado de la calidad: cumplido en el laboratorio. Con YOLO26n reentrenado a 1280 px, las personas llegan a 88,8 % a 5 FPS, 90,9 % a 10 FPS y 86,4 % a 2 FPS, con una inferencia por cuadro. El modelo de línea base, inferido a 1280 px, se quedaba en 84,0 % a 5 FPS.

Estado de la velocidad: pendiente de la medición en placa. El requisito de 5 FPS se mantiene por la distancia entre detecciones; la calidad ya no depende de él, porque a 2 FPS se detecta el 86,4 % de las personas.

6.3. Detección por objeto durante la pasada

Definición. Para cada clase de la misión, es la fracción de objetos únicos (identificados por su target_id en VisDrone-VID) que en al menos uno de los cuadros evaluados tiene una detección de la clase correcta con IoU de al menos 0,5.

Cálculo. En cada cuadro se emparejan detecciones y anotaciones igual que en la evaluación por cuadro. El target_id de cada anotación emparejada marca a ese objeto como detectado. No hace falta un tracker para medir; el tracker hace falta recién en vuelo, para confirmar y contar objetos.

Indicadores que la acompañan, según el objetivo de detección por pasada:

Indicador	Para qué
Detección por objeto (al menos 1 cuadro)	Métrica principal frente al 85 %
Detección por objeto (al menos 3 cuadros)	Lo que necesita un tracker para confirmar un objeto
Falsas alarmas por cuadro y por minuto	Evita que un umbral bajo infle el resultado
Recall y precisión por cuadro	Comparación directa con la evaluación en imágenes fijas
Tiempo hasta la primera detección	Cuánto tarda el sistema en reportar un objeto nuevo
Desglose por tamaño máximo	Qué objetos quedan fuera del alcance de la resolución

Tabla 12. Indicadores de la evaluación en video.

Cada indicador se calcula submuestreando el video a los FPS simulados de cada placa (por ejemplo, 30, 10, 5 y 2 FPS), usando los resultados del benchmark en placas.

Estado: **implementada** en `tp4/video.py` (`python -m tp4.cli video runs/full/ID`), con el emparejamiento y el agrupamiento de `tp4/mission.py`. Las detecciones que caen dentro de regiones ignoradas no cuentan como falsas alarmas. Resultados en la evaluación en video.

7. Mejoras de la inferencia

7.1. Resolución de entrada y mosaicos

7.1.1. Hipótesis

La inferencia inicial procesa la imagen completa reducida a 640 px de lado. Las imágenes de DET miden entre 1360 y 1920 px de ancho, y los videos de VID llegan a 3840 px, así que se reducen entre 2 y 6 veces. En DET, el 46 % de las personas queda por debajo de 8 px⁸. Si el recall depende sobre todo del tamaño aparente, aumentar la resolución efectiva debería recuperar gran parte de esas personas.

7.1.2. Alternativas

Alternativa	Qué hace	Costo por imagen
Entrada de 960 o 1280 px	Reduce menos la imagen: los objetos quedan 1,5 o 2 veces más grandes	Unas 2,25 o 4 veces más píxeles para la red
Mosaicos (SAHI)	Corta la imagen nativa en mosaicos solapados de 640 px y une las detecciones [8]	Una inferencia por mosaico: unas 6 en DET y hasta 33 en video 4K
Mosaicos reducidos	Reduce la imagen a 1920 px de lado mayor antes de cortarla	Hasta 9 inferencias por cuadro, también en 4K
Mosaicos + 1280 px	Mosaicos nativos con la pasada completa a 1280 px	Como los mosaicos nativos
Volar más bajo	Agranda los objetos sin cambiar el software	Cubre menos terreno por pasada

Tabla 13. Palancas para recuperar objetos pequeños.

7.1.3. Resultado sin reentrenar

La evaluación de resolución confirma la hipótesis en DET⁹:

- Con **1280 px**, el recall de personas por cuadro pasa de 27 % a 43 % y el mAP50 de la misión, de 48,4 % a 61,4 %, con una sola inferencia.
- Con **mosaicos**, el recall de personas llega a 45 % y el de todos los objetos a 61 %, pero con menos precisión y varias inferencias por imagen.
- La estimación previa de alrededor de 50 % de recall de personas por cuadro fue optimista.
- Combinando mosaicos con más resolución, en DET el recall de personas por cuadro llega a 48 % (mosaicos nativos + 1280 px).

En video, frente al objetivo del 85 % por pasada a 5 FPS, las personas llegan a 84 % con 1280 px, 88 % con mosaicos sobre la imagen reducida a 1920 px (8,5 inferencias por cuadro) y 89 % con mosaicos nativos + 1280 px (unas 24 inferencias por cuadro). Ver la evaluación en video.

⁸`mission_eval/val.json`, recall por tamaño.

⁹`mission_eval/resolution_val.json` de la ejecución YOLO26n.

7.1.4. Próximos pasos

1. **Medir el costo en las placas.** Agregar 1280 px y mosaicos reducidos a 1920 px (y mosaicos nativos + 1280 px en la Jetson) al benchmark: en una placa el costo crece cerca de la cantidad de píxeles, no como en la GPU de la laptop.
2. **Reentrenar una sola vez** con las dos clases de la misión y entrada de 1280 px. **Hecho:** recuperó las personas grandes (de 65 % a 86 % de recall) y llevó la detección por pasada a 89 % a 5 FPS; ver reentrenamiento.
3. **Considerar un modelo híbrido** si la placa no alcanza: 1280 px solo en zonas de interés o mosaicos a menor frecuencia que la pasada completa.

7.2. Evaluación de resolución y mosaicos

Esta evaluación corresponde al modelo de **línea base**: YOLO26n entrenado a 640 px con las diez clases de VisDrone. Los resultados del modelo reentrenado con persona y vehículo a 1280 px están en el reentrenamiento.

7.2.1. Método

Se evaluó el mismo `best.pt` de YOLO26n, **sin reentrenar**, sobre las 548 imágenes de validación de DET, con las clases de la misión y cuatro formas de procesar cada imagen¹⁰:

- **640, 960 y 1280 px:** la imagen completa reducida a ese lado mayor.
- **Mosaicos:** además de la pasada completa a 640 px, la imagen nativa se corta en mosaicos de 640 px con 20 % de solapamiento, al estilo de SAHI [8]. Se descartan las cajas cortadas por un borde interno del mosaico y se unen las detecciones con supresión por clase (IoU 0,5).
- **Mosaicos + 1280 px:** los mismos mosaicos nativos, pero con la pasada completa a 1280 px en lugar de 640 px.

La variante de mosaicos sobre una imagen reducida (1920 o 2560 px de lado mayor) coincide en DET con los mosaicos nativos, porque sus imágenes no superan los 1920 px; se evalúa en video.

Todas las variantes usan el pipeline `predict` con confianza mínima 0,01, que subestima levemente el mAP frente al validador; la variante de 640 px sirve de referencia con el mismo pipeline. El tamaño de cada objeto se refiere siempre a la entrada de 640 px, para comparar las mismas personas entre variantes.

7.2.2. Resultados

Variante	Inferencias por imagen	ms por imagen (RTX 5070)	mAP50	mAP50-95	Recall persona	Recall todos
640 px	1	16,2	48,4 %	25,1 %	27,2 %	44,5 %
960 px	1	17,0	57,0 %	30,9 %	37,4 %	53,8 %
1280 px	1	18,5	61,4 %	33,6 %	42,9 %	57,6 %
Mosaicos	6,2	53,9	55,8 %	31,7 %	45,2 %	61,3 %
Mosaicos + 1280 px	6,2	60,8	61,0 %	33,6 %	48,0 %	63,1 %

Tabla 14. Variantes de resolución con las clases de la misión en DET val. Recall por cuadro con confianza 0,25 e IoU 0,5 (`mission_eval/resolution_val.json` de la ejecución YOLO26n).

¹⁰`tp4/inference.py`; comando `python -m tp4.cli resolution runs/full/20260925T231501Z`.

Variante	persona P / R	vehículo P / R
640 px	66,3 % / 27,2 %	76,6 % / 54,3 %
960 px	63,9 % / 37,4 %	76,5 % / 63,0 %
1280 px	64,8 % / 42,9 %	77,8 % / 65,9 %
Mosaicos	64,2 % / 45,2 %	71,3 % / 70,4 %
Mosaicos + 1280 px	64,5 % / 48,0 %	73,7 % / 71,6 %

Tabla 15. Precisión (P) y recall (R) por clase con confianza 0,25.

Variante	< 4 px	4–8 px	8–16 px	16–32 px	≥ 32 px
640 px	0,4 %	11,3 %	37,5 %	61,5 %	76,1 %
960 px	3,5 %	23,1 %	49,2 %	66,7 %	66,1 %
1280 px	9,8 %	29,6 %	55,2 %	66,3 %	65,1 %
Mosaicos	11,5 %	33,3 %	56,4 %	67,5 %	78,0 %
Mosaicos + 1280 px	14,0 %	36,8 %	59,4 %	68,2 %	67,9 %

Tabla 16. Recall de personas según su tamaño en la entrada de 640 px, confianza 0,25.

7.2.3. Lectura

- **La hipótesis se confirma:** duplicar la resolución efectiva recupera personas pequeñas. Entre 4 y 8 px, el recall pasa de 11 % a 30–33 %. La estimación previa de alrededor de 50 % de recall de personas fue optimista: se llega a 43–45 %.
- **1280 px es el mejor compromiso en precisión global:** mayor mAP con una sola inferencia. Sus personas grandes (≥ 32 px) bajan de 76 % a 65 %, porque quedan más grandes que lo que el modelo vio al entrenar a 640 px.
- **Mosaicos + 1280 px da el mayor recall por cuadro** (48 % de personas, 63 % de todos los objetos) con casi el mAP de 1280 px, con un costo similar al de los mosaicos con pasada a 640 px. Como 1280 px, pierde algo en personas grandes.
- **Los mosaicos con pasada a 640 px maximizan el recall de personas grandes**, pero pierden precisión en vehículos, por falsas detecciones en las uniones, y cuestan unas 6 inferencias por imagen de VisDrone, y muchas más en video 4K.
- **El tiempo de la laptop no predice el de la placa.** En la RTX 5070, 1280 px cuesta apenas un 15 % más que 640 px porque la GPU está subutilizada con lote 1. En una placa el costo crece más cerca de la cantidad de píxeles: alrededor de 4 veces para 1280 px. Por eso estas variantes deben pasar por el benchmark en placas.
- **Reentrenar a mayor resolución** podría recuperar las personas grandes y sumar recall, ahora con una base medida contra la cual comparar.

8. Detección por pasada en video

Esta evaluación corresponde al modelo de **línea base**: YOLO26n entrenado a 640 px con las diez clases de VisDrone. Los resultados del modelo reentrenado con persona y vehículo a 1280 px están en el reentrenamiento.

8.1. Método

Se procesaron los 2846 cuadros de las siete secuencias de validación de VisDrone-VID con el mismo `best.pt` de YOLO26n, sin reentrenar, en tres variantes de la evaluación de resolución: imagen completa a 640 px, a 1280 px y mosaicos nativos¹¹. Sobre esas predicciones se calculó la detección por objeto con las reglas del objetivo por pasada:

- un objeto cuenta como detectado si en algún cuadro evaluado tiene una detección de su clase de la misión con IoU de al menos 0,5;
- para simular los FPS de una placa, se evalúa uno de cada k cuadros (30 FPS nominales: k = 1, 3, 6 y 15 equivalen a 30, 10, 5 y 2 FPS);
- las falsas alarmas excluyen las detecciones que caen dentro de regiones ignoradas.

Por la fuga con el entrenamiento de DET, las cifras principales usan solo las **tres secuencias limpias** (086, 117 y 268). Allí hay 187 personas y 89 vehículos (incluidas 10 bicicletas y motos).

8.2. Detección al menos una vez

Variante	FPS simulados	persona	vehículo
640 px	30	80,0 %	56,2 %
640 px	10	75,9 %	55,1 %
640 px	5	74,3 %	52,8 %
640 px	2	65,2 %	52,8 %
1280 px	30	86,3 %	64,0 %
1280 px	10	85,0 %	64,0 %
1280 px	5	84,0 %	64,0 %
1280 px	2	80,4 %	62,9 %
Mosaicos	30	89,5 %	75,3 %
Mosaicos	10	87,7 %	73,0 %
Mosaicos	5	87,2 %	71,9 %
Mosaicos	2	77,7 %	65,2 %

Tabla 17. Objetos detectados al menos una vez en las secuencias limpias, con confianza 0,25 (`mission_eval/video_val.json` de la ejecución YOLO26n).

Variante	persona limpias / todas	vehículo limpias / todas
640 px	74,3 % / 79,2 %	52,8 % / 81,2 %
1280 px	84,0 % / 87,5 %	64,0 % / 86,5 %
Mosaicos	87,2 % / 90,5 %	71,9 % / 89,8 %

Tabla 18. Efecto de la fuga a 5 FPS y confianza 0,25. "Todas" incluye las cuatro secuencias cuyo video aporta imágenes al entrenamiento de DET.

¹¹`tp4/video.py`; comando `python -m tp4.cli video runs/full/20260925T231501Z`.

8.3. Personas: umbral, confirmación y falsas alarmas

Variante	Confianza	Detectada ≥ 1	Detectada ≥ 3	Recall por cuadro	Precisión por cuadro	Falsas alarmas por cuadro
640 px	0,1	83,4 %	74,3 %	61,7 %	35,4 %	21,4
640 px	0,25	74,3 %	59,9 %	47,9 %	64,4 %	5,0
640 px	0,4	56,7 %	42,2 %	34,2 %	81,7 %	1,5
1280 px	0,1	90,9 %	81,8 %	67,8 %	36,6 %	22,3
1280 px	0,25	84,0 %	72,2 %	55,9 %	60,3 %	7,0
1280 px	0,4	73,3 %	63,1 %	45,9 %	74,8 %	2,9
Mosaicos	0,1	92,5 %	84,5 %	68,1 %	30,4 %	29,7
Mosaicos	0,25	87,2 %	72,7 %	57,3 %	55,1 %	8,9
Mosaicos	0,4	76,5 %	62,0 %	45,8 %	70,5 %	3,6

Tabla 19. Personas en las secuencias limpias a 5 FPS simulados.

Variante	< 4 px	4–8 px	8–16 px	16–32 px	≥ 32 px
640 px	0,0 %	10,0 %	74,3 %	90,4 %	100,0 %
1280 px	0,0 %	35,0 %	86,5 %	95,9 %	100,0 %
Mosaicos	75,0 %	65,0 %	86,5 %	91,8 %	100,0 %

Tabla 20. Personas detectadas al menos una vez según su tamaño máximo en la pasada, referido a 640 px (4, 20, 74, 73 y 16 personas por columna), a 5 FPS y confianza 0,25.

8.4. Combinaciones de mosaicos y resolución

Se probaron dos combinaciones: **mosaicos + 1280 px** (mosaicos nativos con la pasada completa a 1280 px) y **mosaicos reducidos** (pasada completa a 640 px y mosaicos sobre la imagen reducida a 1920 o 2560 px de lado mayor, solo si es más grande). Reducir antes de cortar baja el costo en los videos grandes: en 4K, 9 o 16 inferencias por cuadro en lugar de 33.

Variante	Inferencias por cuadro	ms por cuadro	Persona a 30 FPS	Persona / vehículo a 5 FPS	Persona / vehículo a 2 FPS	Falsas personas por cuadro
640 px	1,0	18	80,0 %	74,3 % / 52,8 %	65,2 % / 52,8 %	5,0
1280 px	1,0	19	86,3 %	84,0 % / 64,0 %	80,4 % / 62,9 %	7,0
Mosaicos reducidos a 1920 px	8,5	71	92,6 %	88,2 % / 65,2 %	81,5 % / 65,2 %	9,1
Mosaicos reducidos a 2560 px	13,7	112	91,6 %	87,7 % / 66,3 %	78,8 % / 65,2 %	9,6
Mosaicos nativos (640 px)	23,5	131	89,5 %	87,2 % / 71,9 %	77,7 % / 65,2 %	8,9
Mosaicos nativos + 1280 px	23,5	148	91,6 %	89,3 % / 71,9 %	84,2 % / 66,3 %	8,9

Tabla 21. Objetos detectados al menos una vez en las secuencias limpias según la variante, con confianza 0,25. Inferencias y tiempos promediados por cuadro en esas secuencias (RTX 5070); falsas alarmas a 5 FPS.

- **Mosaicos reducidos a 1920 px es el mejor compromiso para personas:** 88 % a 5 FPS con 8,5 inferencias por cuadro, casi lo mismo que la combinación más cara (89 %) con un tercio del costo. Supera el 85 % a 5 FPS, que 1280 px sola no alcanza (84 %).
- **Mosaicos nativos + 1280 px es la más completa:** es la única que se sostiene a 2 FPS (84 % de personas) y, junto con los mosaicos nativos, la mejor en vehículos (72 %), pero con unas 24 inferencias por cuadro, y 33 en 4K.
- **Reducir la imagen antes de cortar sacrifica los vehículos chicos del 4K** (de 72 % a 65–66 %), que solo se recuperan cortando a resolución nativa.
- **Las falsas alarmas suben con cualquier combinación**, a unas 9 personas falsas por cuadro, frente a 7 con 1280 px.

8.5. Lectura

- **El objetivo por pasada es alcanzable para personas.** A 5 FPS y confianza 0,25, las personas detectadas al menos una vez suben de 74 % (640 px) a 84 % (1280 px) y 87 % (mosaicos). A 10 FPS, 1280 px llega a 85 %. Con 640 px el objetivo no se alcanza en ningún escenario limpio.
- **La pasada multiplica las oportunidades.** El recall por cuadro de personas es 56–57 %, pero cada persona aparece en decenas de cuadros. La mediana del tiempo hasta la primera detección es 0 s: casi siempre se detectan en su primer cuadro evaluado. Bajar de 5 a 2 FPS cuesta entre 4 y 10 puntos.
- **Los vehículos pequeños en 4K no se alcanzan.** En la secuencia 4K (268), 29 de 46 vehículos no se detectan nunca a 1280 px, aunque son visibles durante 38 a 263 cuadros: miden entre 7 y 15 px referidos a 640. Los mosaicos nativos recuperan parte, y aun así vehículo queda en 72 %. Ese escenario pide volar más bajo o reentrenar con más resolución.
- **El costo son las falsas alarmas.** Con confianza 0,25 hay entre 5 y 9 falsas personas por cuadro en escenas densas, y la precisión por cuadro ronda el 55–64 %, similar a DET. Exigir 3 detecciones antes de confirmar un objeto baja la detección de personas a 72–73 % con 1280 px o mosaicos. Un tracker que confirme por persistencia debería filtrar la mayoría de esas falsas alarmas, que no se sostienen entre cuadros, pero eso no está medido.
- **La fuga infla los resultados,** sobre todo en vehículos: a 5 FPS con 640 px pasan de 53 % en limpias a 81 % con todas las secuencias. Las conclusiones se apoyan en las secuencias limpias.
- **Costo en la laptop:** unos 15–19 ms por cuadro a 640 px, 16–23 ms a 1280 px y 52–166 ms con mosaicos (hasta 33 inferencias por cuadro en 4K). El costo en la placa se mide en el benchmark.

9. Reentrenamiento con las clases de la misión

9.1. Qué se cambió

Las evaluaciones de la línea base mostraron que el límite era el tamaño aparente de las personas, y que inferir a 1280 px con un modelo entrenado a 640 px recuperaba las pequeñas pero perdía las grandes (ver resolución y mosaicos). Por eso YOLO11n y YOLO26n se entrenaron de nuevo con dos cambios respecto del protocolo de la línea base:

- **Clases de la misión desde las etiquetas.** Cada etiqueta de VisDrone se convierte a persona o vehículo al preparar los datos (ver clases de la misión). El modelo, la evaluación y las placas usan las mismas dos clases.
- **Entrada de 1280 px,** la configuración por defecto de la misión.

Las épocas, el lote, la semilla y los datos son los mismos. YOLO11n tardó 2,51 h y YOLO26n, 2,93 h¹².

9.2. Validación

Modelo	Precision	Recall	mAP50	mAP50–95	AP50 persona	AP50 vehículo
YOLO11n	77,6 %	62,1 %	69,7 %	39,7 %	59,2 %	80,3 %
YOLO26n	79,4 %	62,3 %	70,4 %	40,0 %	60,0 %	80,8 %

Tabla 22. Evaluación de best.pt con el validador de Ultralytics sobre las 548 imágenes de validación, a 1280 px (run.json de 20260927T215941Z (YOLO11n) y 20260928T003017Z (YOLO26n)). Estas cifras de dos clases no se comparan con el mAP de diez clases de la línea base.

9.3. Comparación con la línea base

La comparación usa la misma variante (imagen completa a 1280 px) y el mismo procedimiento de predicción para los tres modelos. La línea base es YOLO26n entrenado a 640 px con diez clases, inferido a 1280 px y reagrupado¹³.

Modelo	mAP50 misión	Recall persona	Recall vehículo	Precisión persona
Línea base (YOLO26n, 640 px, 10 clases)	61,4 %	42,9 %	65,9 %	64,8 %
YOLO11n reentrenado	69,3 %	57,3 %	75,7 %	63,1 %
YOLO26n reentrenado	69,9 %	57,4 %	75,8 %	64,5 %

Tabla 23. DET val con imagen completa a 1280 px. Recall y precisión por cuadro con confianza 0,25 e IoU 0,5 (mission_eval/resolution_val.json de cada ejecución; comando python -m tp4.cli resolution).

Tamaño de la persona	< 4 px	4–8 px	8–16 px	16–32 px	≥ 32 px
Línea base	9,8 %	29,6 %	55,2 %	66,3 %	65,1 %
YOLO26n reentrenado	15,3 %	45,4 %	69,7 %	82,2 %	86,2 %

Tabla 24. Recall de personas por cuadro según su tamaño, referido a la entrada de 640 px, con imagen completa a 1280 px y confianza 0,25.

¹²runs/full/20260928T003017Z/run.json.

¹³runs/full/20260925T231501Z/mission_eval/resolution_val.json y video_val.json.

La mejora aparece en todos los tamaños. En las personas grandes, la línea base caía de 76 % (a 640 px) a 65 % al inferir a 1280 px, porque quedaban más grandes que las vistas en entrenamiento; el modelo reentrenado llega a 86 %.

Modelo	Personas a 10 FPS	Personas a 5 FPS	Personas a 2 FPS	Vehículos a 5 FPS	Falsas personas por cuadro
Línea base	85,0 %	84,0 %	80,4 %	64,0 %	7,0
YOLO11n reentrenado	90,4 %	89,8 %	85,9 %	65,2 %	8,0
YOLO26n reentrenado	90,9 %	88,8 %	86,4 %	65,2 %	6,9

Tabla 25. Objetos detectados al menos una vez durante la pasada en las tres secuencias limpias de VisDrone-VID (187 personas y 89 vehículos), imagen completa a 1280 px, confianza 0,25 (mission_eval/video_val.json de cada ejecución; comando python -m tp4.cli video).

En la demostración en video del notebook, sobre la secuencia uav0000117_02622_v, a 10 FPS simulados, YOLO26n reentrenado detecta al menos una vez 93 de 105 personas, frente a 84 de la línea base.

9.4. Resolución de entrada con el modelo reentrenado

Variante	Inferencias por imagen	mAP50 misión	Recall persona	Recall vehículo	Personas por pasada a 5 FPS
Imagen completa 640 px	1	49,9 %	33,4 %	58,9 %	78,6 %
Imagen completa 960 px	1	63,2 %	49,5 %	70,7 %	—
Imagen completa 1280 px	1	69,9 %	57,4 %	75,8 %	88,8 %
Mosaicos nativos	6,2	64,7 %	59,5 %	77,8 %	93,0 %
Mosaicos nativos + 1280 px	6,2	70,0 %	62,3 %	79,6 %	95,2 %

Tabla 26. YOLO26n reentrenado. mAP y recall en DET val con confianza 0,25; detección por pasada en las secuencias limpias de VID. En video, por la secuencia 4K, los mosaicos nativos cuestan unas 23,5 inferencias por cuadro; sobre la imagen reducida a 1920 px, 8,5, con el mismo 93,0 % de personas.

- **El modelo necesita la entrada de 1280 px.** Inferido a 640 px pierde casi toda la mejora (49,9 % de mAP50, frente a 48,4 % de la línea base a 640 px). Si una placa no sostiene 1280 px, no alcanza con bajar la resolución de este modelo.
- **Los mosaicos agregan recall a un costo alto.** Sobre la imagen completa a 1280 px suman unos 5 puntos de recall de personas y casi no cambian el mAP, con seis o más inferencias por imagen. Siguen siendo un resultado de laboratorio.
- En DET val, los mosaicos sobre la imagen reducida a 1920 o 2560 px coinciden con los nativos: ninguna imagen de validación supera los 1920 px.

9.5. Umbral de confianza

Confianza	Línea base: personas / falsas por cuadro	YOLO26n reentrenado: personas / falsas por cuadro
0,10	90,9 % / 22,3	94,1 % / 19,2
0,25	84,0 % / 7,0	88,8 % / 6,9
0,40	73,3 % / 2,9	84,0 % / 3,4

Tabla 27. Personas detectadas al menos una vez por pasada a 5 FPS y falsas personas por cuadro, imagen completa a 1280 px, secuencias limpias.

Con el modelo reentrenado, subir la confianza a 0,4 baja las falsas alarmas a la mitad y deja la detección por pasada en 84 %, casi en el objetivo. Es un insumo para la decisión abierta sobre el umbral operativo.

9.6. Referencia en test-dev

El paquete del benchmark lleva YOLO26n reentrenado y su evaluación de referencia en la laptop sobre las 300 imágenes de test-dev, a 1280 px: mAP50 57,5 % y mAP50–95 32,0 %, con AP50 de 35,0 % en persona y 80,0 % en vehículo¹⁴. Las personas de test-dev resultan mucho más difíciles que las de validación (AP50 de 60,0 %). No es la evaluación final de test-dev: las vistas previas por época observaron imágenes de ese conjunto.

9.7. Conclusiones

- **El objetivo se cumple en el laboratorio con una sola inferencia por cuadro.** YOLO26n reentrenado a 1280 px detecta al menos una vez el 88,8 % de las personas a 5 FPS y el 86,4 % a 2 FPS, sin más falsas alarmas que la línea base.
- **Reentrenar corrige el desajuste de escala.** Mejora el recall de personas pequeñas y recupera las grandes que la línea base perdía al inferir a 1280 px.
- **YOLO11n y YOLO26n vuelven a ser equivalentes en detección.** YOLO26n comete menos falsas alarmas y no requiere NMS, por lo que sigue siendo el modelo de las placas.
- **Los vehículos pequeños del video 4K siguen sin alcanzarse** (65 % por pasada). Requieren volar más bajo o mosaicos a resolución nativa.
- **La pregunta pasa a ser el hardware:** qué FPS sostiene cada placa con YOLO26n a 1280 px.

10. Benchmark en computadoras de a bordo

10.1. Plataformas y variantes

	NVIDIA Jetson (con GPU)	Raspberry Pi 5 (sin GPU de cómputo)
Procesador	SoC ARM64 con GPU integrada CUDA; modelo exacto a registrar por el equipo	SoC ARM64, CPU de 4 núcleos Cortex-A76
Memoria	Compartida entre CPU y GPU	RAM del sistema
Referencia	PyTorch FP32 sobre CUDA	PyTorch FP32 sobre CPU
Optimizado	TensorRT FP16 [9], motor construido en la propia placa	NCNN [10] [11], 4 hilos, Vulkan desactivado
Telemetría	tegrastats, temperaturas, frecuencias	vcgencmd get_throttled, temperaturas, frecuencias
Guía de referencia	Ultralytics para Jetson [12]	Ultralytics para Raspberry Pi [13]

Tabla 28. Configuraciones que se medirán. La comparación principal es original frente a optimizado dentro de cada placa; entre placas se comparan soluciones completas, con hardware, motor y precisión numérica diferentes.

10.2. Benchmark en placas

10.2.1. Preguntas de investigación

1. ¿Cuánto acelera el motor optimizado (TensorRT FP16 o NCNN) respecto de PyTorch *en la misma placa*?
2. ¿Cuánta precisión se pierde al exportar y cambiar la precisión numérica (Δ mAP respecto del .pt en la misma placa)?

¹⁴BENCHMARK_OBC.md; paquete package_20260928T020732595419Z.

3. ¿Se sostiene el rendimiento durante cinco minutos continuos, o aparece limitación térmica (*throttling*)?
4. ¿Qué huella de memoria deja cada variante para el resto del software de a bordo?
5. Con GPU y sin GPU, ¿qué configuración satisface el requisito operativo y el objetivo de detección por pasada?

10.2.2. Paquete común y reproducibilidad

Ambos equipos reciben **el mismo ZIP** (unos 61 MiB) con su archivo `.sha256`¹⁵. Contiene:

- 300 imágenes únicas de test-dev, elegidas sin reemplazo con semilla 42, junto con sus etiquetas y un orden fijo;
- el `best.pt` de YOLO26n reentrenado con persona y vehículo a 1280 px (ejecución 20260928T003017Z), con su manifiesto de entrenamiento (**no** debe reemplazarse por pesos descargados de Internet);
- doce imágenes de galería, la configuración y el SHA-256 de cada archivo;
- la evaluación de referencia en la laptop sobre esa misma muestra, a 1280 px y con las dos clases: **mAP50 = 57,52 %** y **mAP50-95 = 32,02 %** (persona 35,0 % y vehículo 80,0 % de AP50).

10.2.3. Cadena medida

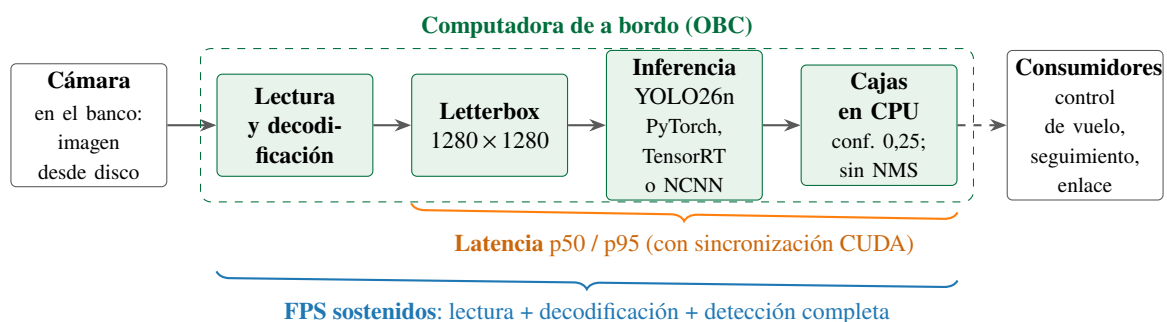


Figura 5. Procesamiento a bordo y los intervalos que mide el benchmark. En vuelo, la cámara reemplaza la lectura desde disco; el dibujo y el guardado de imágenes quedan fuera de la medición.

10.2.4. Protocolo de medición

- **Repeticiones:** tres corridas de cinco minutos por modelo, cada una con 50 imágenes de calentamiento. El orden alterna entre PyTorch/optimizado, optimizado/PyTorch y PyTorch/optimizado, con pausas de 60 s entre procesos para limitar el sesgo térmico.
- **Aislamiento:** cada medición corre en un proceso nuevo que carga solo su variante. Nunca se exporta dentro del proceso medido.
- **Entrada:** una imagen por vez desde disco, lote 1, *letterbox* de 1280 px (la configuración por defecto de la misión), confianza 0,25 y un máximo de 300 detecciones. No se impone NMS a la salida *end-to-end* de YOLO26.

¹⁵BENCHMARK_OBC.md, secciones 1 y 8.

- **Memoria y telemetría:** RSS antes y después de cargar el modelo, y pico muestreado cada 200 ms, junto con temperaturas, frecuencias y estado de *throttling*. En Jetson la memoria es compartida, por lo que no deben sumarse contadores de CPU y GPU.
- **Condiciones:** misma fuente, refrigeración y modo de potencia para ambos modelos, registrados en cada ejecución. No se modifican `nvpmodel` ni los relojes entre mediciones.
- **Precisión:** tras las seis mediciones, procesos separados calculan mAP50, mAP50–95 y el desglose por clase (confianza 0,001) sobre las 300 imágenes, y generan los paneles anotados.

Los indicadores resultantes y su lectura están en rendimiento en placa.

10.2.5. Flujo de trabajo

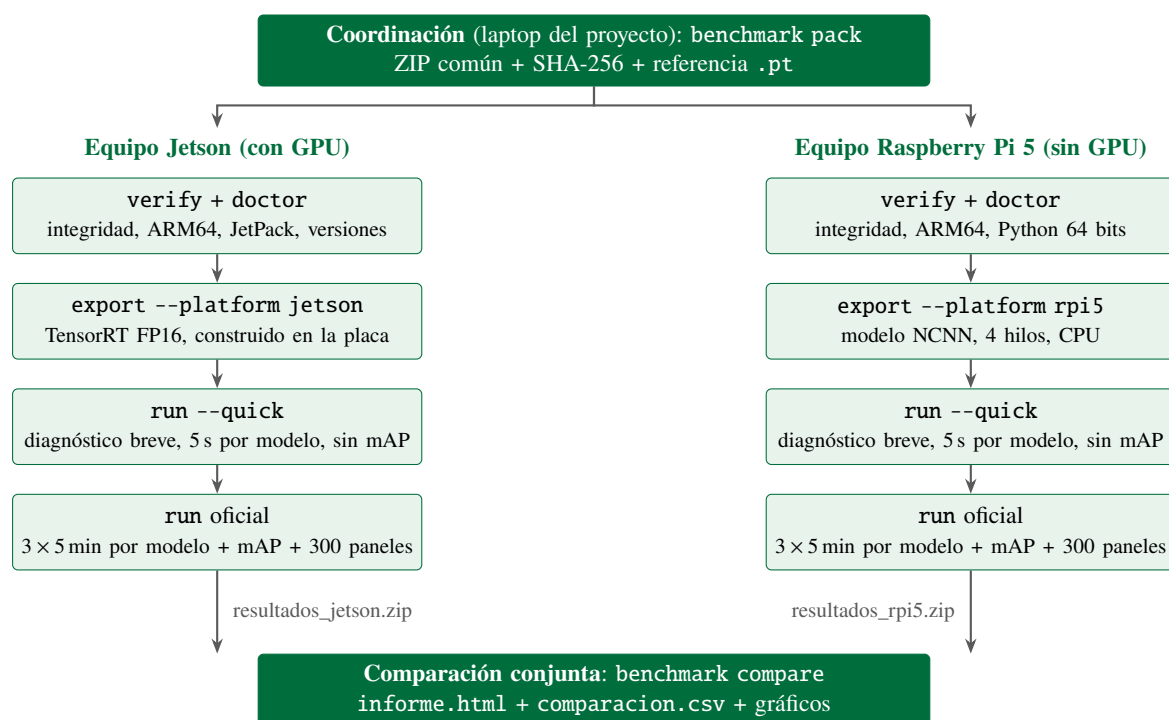


Figura 6. Flujo de trabajo del benchmark (comandos de python `-m tp4.benchmark`). El detalle operativo está en `BENCHMARK_OBC.md`.

10.2.6. Estado de preparación

Tarea	Estado
Paquete de YOLO26n reentrenado a 1280 px generado (package_20260928T020732595419Z) y verificado por CRC y hashes; el código rechaza paquetes con otro checkpoint u otras clases	Hecho
Selección de las 300 imágenes reproducible con semilla 42	Hecho
Referencia .pt en la laptop a 1280 px (mAP50 57,52 %; mAP50–95 32,02 %)	Hecho
Nueve pruebas automatizadas (integridad, checkpoint y clases, rutas, percentiles, rechazos, exclusión de diagnósticos, reporte)	Aprobadas
Inferencia real en CPU y anotación en Windows, marcadas como diagnóstico local	Hecho
Reporte HTML sin resultados abierto sin conexión de red	Hecho
Exportación TensorRT / NCNN y mediciones físicas en las placas	Pendiente

Tabla 29. Validación local realizada el 26 de septiembre de 2026; paquete regenerado el 28. Ninguna cifra de la laptop se atribuye a las placas.

10.3. Indicadores de rendimiento en placa

Indicador	Interpretación para el dron
FPS sostenidos (media y dispersión)	Frecuencia real de detección; se contrasta con el requisito $\Delta d = v/f$.
Latencia p50 / p95	Retardo típico y en el peor caso habitual entre la captura y la decisión.
Factor de aceleración	Ganancia del motor optimizado frente al .pt en la misma placa.
Δ mAP frente al .pt	Costo en precisión de exportar y cambiar la precisión numérica.
RSS pico y ahorro de RSS	Memoria disponible para navegación, telemetría y registro.
Temperatura y <i>throttling</i>	Estabilidad durante el vuelo; necesidad de disipación (peso adicional).

Tabla 30. Indicadores del benchmark y su lectura operativa (BENCHMARK_OBC.md, secciones 5 y 7).

El protocolo actual no mide consumo energético: temperatura, RAM y modo de potencia no equivalen a energía por inferencia. Medirla es una de las decisiones abiertas.

11. Plan de trabajo

11.1. Tareas y criterios de aceptación

Ya se completaron la evaluación por clases de la misión, la de resolución y mosaicos, la de detección por pasada en VisDrone-VID y el reentrenamiento con las dos clases a 1280 px (tarea 8), que cambia el modelo del benchmark. La tabla ordena el trabajo pendiente. Cada tarea tiene un entregable verificable, de modo que cualquier integrante pueda comprobar si está terminada sin depender de quien la ejecutó.

#	Tarea	Responsable	Criterio de aceptación
1	Publicar el código del benchmark y distribuir el ZIP y su .sha256	Coordinación	Ambos equipos trabajan sobre el mismo <i>commit</i> y el checksum verifica.
2	Preparar el entorno y generar el diagnóstico	Cada equipo	<code>doctor.json</code> con ARM64, Python de 64 bits y versiones; en Jetson, versión de L4T/JetPack.
3	Exportar el modelo optimizado	Cada equipo	El motor se construye en la propia placa y pasa la prueba de carga e inferencia.
4	Ejecutar la prueba corta (<code>--quick</code>)	Cada equipo	Termina con <code>status: finished</code> .
5	Ejecutar el benchmark oficial	Cada equipo	Seis mediciones completas, 300 paneles por modelo, telemetría presente, fuente y refrigeración registradas.
6	Comparación conjunta	Coordinación	<code>informe.html</code> se abre en otra computadora sin red; diagnósticos e intentos incompletos excluidos.
7	Agregar 1280 px y mosaicos al benchmark de placas	Coordinación y cada equipo	FPS, latencia p95 y memoria de cada variante en Jetson y Raspberry Pi 5, con el mismo protocolo.
8	Reentrenar con las dos clases de la misión y entrada de 1280 px	Coordinación	Hecho. Comparado con la evaluación a 1280 px sin reentrenar, en DET val y en las secuencias limpias de VID: ver reentrenamiento.
9	Ampliar la evaluación en video sin fuga	Coordinación	Secuencias de VID de test-dev cuyo video no esté en el entrenamiento de DET, para tener más objetos limpios.
10	Evaluar test-dev completo con el modelo fijado	Coordinación	Evaluación única, sin ajustar hiperparámetros después de verla.
11	Análisis e informe final	Todo el equipo	Discusión frente al objetivo de detección por pasada y al requisito operativo.

Tabla 31. Plan de trabajo. Cada ejecución oficial del benchmark requiere al menos 35 minutos, más exportación, calentamientos y evaluación.

11.1.1. Reglas para que los resultados sean comparables

- No reentrenar ni modificar checkpoints o hiperparámetros a partir de los resultados del benchmark.
- No instalar ruedas de PyTorch para x86/Windows en la Jetson ni actualizar drivers o JetPack automáticamente. Si no existe una combinación compatible con Ultralytics 8.4.162, informar el

bloqueo en lugar de cambiar de motor.

- No reemplazar CUDA por CPU de forma silenciosa. Ante un error, corregir el entorno y lanzar una ejecución nueva, sin mezclar repeticiones de intentos distintos.
- Una diferencia de mAP respecto del .pt debe investigarse como posible efecto de la exportación o de la precisión numérica, no ocultarse.
- Las pruebas de resolución y de video se comparan siempre con las mismas dos clases de la misión.

11.2. Decisiones

11.2.1. Decisiones tomadas

Decisión	Motivo
YOLO26n como modelo de referencia en placa	Precisión equivalente a YOLO11n, también al reentrenarlos (mAP50 de la misión 69,9 % y 69,3 %), con menos falsas alarmas; salida sin NMS y orientación a dispositivos de borde.
Dos clases de la misión: persona y vehículo	Es lo que la misión necesita distinguir; agrupar elimina la confusión <i>car/van</i> . Dos ruedas se suma a vehículo: el conductor ya está anotado como persona y hay muy pocos casos limpios.
Mismas clases en todo el TP	Entrenamiento, evaluación en la laptop y placas usan persona y vehículo: las etiquetas se agrupan al preparar los datos.
Objetivo: 85 % de las personas detectadas al menos una vez por pasada, a 5 FPS o más en la placa	El 85 % por cuadro no es alcanzable con objetos de pocos píxeles; la pasada da varias oportunidades. 5 FPS equivale a 2 m entre detecciones a 10 m/s. El objetivo se fija sobre personas porque los vehículos pequeños en 4K no alcanzan ni reentrenando (65 %).
Probar la resolución antes de reentrenar	El 46 % de las personas mide menos de 8 px en la entrada de 640 px; la prueba sin reentrenar confirmó la mejora.
Reentrenar con persona y vehículo a 1280 px	El modelo de 640 px inferido a 1280 perdía personas grandes. Reentrenado, el recall de personas por cuadro sube de 43 % a 57 % y la detección por pasada a 5 FPS, de 84 % a 89 %, con las mismas falsas alarmas.
Entrada de 1280 px por defecto	Con el modelo reentrenado, mAP50 de la misión de 69,9 % y 89–91 % de personas por pasada a 5–10 FPS con una inferencia por cuadro. El modelo necesita esa entrada: a 640 px pierde casi toda la mejora.
Mosaicos solo como resultado de laboratorio	Con el modelo reentrenado llegan a 93–95 % de personas por pasada a 5 FPS, pero con 8,5 a 23,5 inferencias por cuadro; la imagen completa a 1280 px ya cumple el objetivo.
Demo: .pt frente a optimizado en una placa por SSH	El benchmark ya compara ambas variantes; sus FPS se cruzan con la evaluación en video para ver si se cumple el objetivo.
Informar el video solo con secuencias limpias	Cuatro de las siete secuencias de VID comparten video de origen con el entrenamiento de DET.

Tabla 32. Decisiones tomadas al 28 de septiembre de 2026.

11.2.2. Decisiones abiertas

1. **Asignar hardware y responsables.** Registrar el modelo exacto de Jetson (su generación cambia mucho los resultados), la fuente de alimentación y la refrigeración que se usarían en vuelo.
2. **Fijar el escenario de referencia** de la misión: altura, velocidad, cámara y tipo de misión. De él se derivan el FPS mínimo, la latencia p95 máxima y el tamaño mínimo de objeto exigible (ver requisitos operativos).

3. **Incorporar YOLO11n como control en placa** (extensión opcional). La elección de YOLO26n se apoya en que no necesita NMS; medir YOLO11n con el mismo protocolo confirmaría si esa ventaja aparece en la placa.
4. **Medir el consumo energético** (extensión). En un dron la batería es el recurso crítico. En Jetson pueden aprovecharse los sensores de potencia que informa `tegrastats`; en la Raspberry Pi 5, un medidor USB-C. El indicador sería la energía por inferencia (J/cuadro).
5. **Fijar el umbral de confianza operativo**. Con el modelo reentrenado a 1280 px y 5 FPS, 0,25 detecta el 89 % de las personas por pasada con unas 7 falsas personas por cuadro; 0,4 baja las falsas a 3,4 y deja 84 %; 0,1 llega a 94 % con 19. La elección depende de si un tracker confirma los objetos por persistencia.
6. **Trabajos futuros hacia el sistema completo**: cuantización INT8 con calibración en TensorRT, captura desde cámara real (CSI/USB) en lugar de disco, seguimiento multiobjeto para confirmar detecciones e integración con el controlador de vuelo.

12. Limitaciones y riesgos

- **Una sola semilla por modelo**. Las diferencias entre YOLO11n y YOLO26n carecen de significancia estadística.
- **Evaluador no oficial**. Las métricas son las de Ultralytics, no las del evaluador oficial de VisDrone, que trata las regiones ignoradas de otra forma.
- **Test-dev parcialmente observado**. Las vistas previas por época de los entrenamientos usaron imágenes de test-dev, y el benchmark usa 300 imágenes de ese mismo conjunto. Ninguna decisión de modelo debe basarse en esas observaciones.
- **Muestra de 300 imágenes**. La comparación *relativa* dentro de una misma placa es más robusta que los valores absolutos.
- **FPS desde disco**. Las mediciones no equivalen a los FPS con una cámara real ni incluyen el resto del software de a bordo.
- **Sin medición de energía** en el protocolo base.
- **Compatibilidad de software en Jetson**. La combinación de JetPack, PyTorch, TensorRT y Ultralytics 8.4.162 puede bloquear la exportación. Es el principal riesgo de calendario.
- **Discrepancias sin aislar**. Las validaciones internas por época no coinciden exactamente con la evaluación independiente de `best.pt`, y evaluar con `predict` dio unos 3 puntos menos de mAP50 que con el validador.
- **Fuga entre DET y VID**. Cuatro de las siete secuencias de validación de VID vienen de videos que aportan imágenes al entrenamiento de DET. Sus resultados son optimistas; las conclusiones se apoyan en las tres secuencias limpias, que son pocas (187 personas y 89 vehículos).
- **FPS de los videos asumidos**. Los archivos no traen cuadros por segundo; se asumen 30 FPS nominales para convertir cuadros en segundos y simular los FPS de cada placa.
- **Costo de 1280 px en las placas sin medir**. El modelo reentrenado procesa cuatro veces más píxeles que a 640 px. En la laptop eso casi no se nota, porque la GPU está subutilizada con lote 1; en una placa el costo crece cerca de la cantidad de píxeles.

13. Conclusiones

1. El pipeline de datos, entrenamiento y evaluación del TP4 está completo y es trazable: datos auditados con hash, ejecuciones con configuración registrada, evaluaciones reproducibles por línea de comandos y una wiki que concentra el conocimiento del proyecto.
2. YOLO11n y YOLO26n alcanzan una precisión equivalente sobre VisDrone (mAP50–95 cercano al 16 %). El límite no es la arquitectura, sino el tamaño aparente de los objetos al procesar la imagen a 640 px.
3. Agrupar las clases según la misión elimina la confusión entre vehículos, y reentrenar con persona y vehículo a 1280 px ataca el límite real, el tamaño aparente: frente a la línea base inferida a 1280 px, el mAP50 de la misión pasa de 61,4 % a 69,9 %, y el recall de personas por cuadro, de 43 % a 57 %. El modelo recupera tanto personas pequeñas como las grandes que la línea base perdía al cambiar de escala.
4. Medido por pasada en video y con secuencias limpias, el objetivo del 85 % se cumple para personas con una sola inferencia por cuadro: 89 % a 5 FPS y 86 % a 2 FPS, frente a 84 % y 80 % de la línea base a 1280 px. Para vehículos muy pequeños en video 4K no alcanza (65 %): requiere volar más bajo o mosaicos a resolución nativa.
5. El precio de esa detección son las falsas alarmas: unas 7 falsas personas por cuadro con confianza 0,25, igual que la línea base; con 0,4 bajan a 3,4 y la detección por pasada queda en 84 %. Un tracker que confirme por persistencia es la forma natural de reducirlas y todavía no está medido.
6. YOLO11n y YOLO26n vuelven a ser equivalentes al reentrenarlos; YOLO26n sigue como modelo de las placas por su salida sin NMS y sus menores falsas alarmas. La decisión de despliegue depende ahora del hardware: el modelo necesita la entrada de 1280 px, cuatro veces más píxeles que 640 px, y el benchmark en Jetson (con GPU) y Raspberry Pi 5 (sin GPU) ya tiene el paquete regenerado con este modelo.

Referencias

- [1] T.-Y. Lin *et al.*, “Microsoft COCO: Common Objects in Context”, en *Proc. ECCV*, 2014, pp. 740–755.
- [2] P. Zhu, L. Wen, D. Du, X. Bian, H. Fan, Q. Hu y H. Ling, “Detection and Tracking Meet Drones Challenge”, *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 44, n.º 11, pp. 7380–7399, 2022.
- [3] Ultralytics, *VisDrone Dataset*. [En línea]. Disponible: <https://docs.ultralytics.com/datasets/detect/visdrone/>
- [4] VisDrone, *VisDrone2018-MOT-toolkit: formato de anotaciones y evaluación de seguimiento*. [En línea]. Disponible: <https://github.com/VisDrone/VisDrone2018-MOT-toolkit>
- [5] G. Jocher y J. Qiu, *Ultralytics YOLO11*, 2024. [En línea]. Disponible: <https://github.com/ultralytics/ultralytics>
- [6] J. Redmon, S. Divvala, R. Girshick y A. Farhadi, “You Only Look Once: Unified, Real-Time Object Detection”, en *Proc. IEEE CVPR*, 2016, pp. 779–788.
- [7] Ultralytics, *YOLO26 – Documentación oficial*. [En línea]. Disponible: <https://docs.ultralytics.com/models/yolo26/>
- [8] F. C. Akyon, S. O. Altinuc y A. Temizel, “Slicing Aided Hyper Inference and Fine-tuning for Small Object Detection”, en *Proc. IEEE ICIP*, 2022.

- [9] Ultralytics, *TensorRT Export for YOLO Models*. [En línea]. Disponible: <https://docs.ultralytics.com/integrations/tensorrt/>
- [10] Tencent, *ncnn: high-performance neural network inference framework optimized for the mobile platform*. [En línea]. Disponible: <https://github.com/Tencent/ncnn>
- [11] Ultralytics, *NCNN Export for YOLO Models*. [En línea]. Disponible: <https://docs.ultralytics.com/integrations/ncnn/>
- [12] Ultralytics, *Quick Start Guide: NVIDIA Jetson with Ultralytics YOLO*. [En línea]. Disponible: <https://docs.ultralytics.com/guides/nvidia-jetson/>
- [13] Ultralytics, *Quick Start Guide: Raspberry Pi with Ultralytics YOLO*. [En línea]. Disponible: <https://docs.ultralytics.com/guides/raspberry-pi/>

A. Anexo: trazabilidad de artefactos

Artefacto	Ubicación en tp4-yolo/
Ejecución YOLO11n	runs/full/20260925T015117Z/ (run.json, train/results.csv, latency.json, error_sample.json)
Ejecución YOLO26n (línea base)	runs/full/20260925T231501Z/ (mismos archivos y epoch_previews/)
Reentrenamiento a 1280 px	YOLO11n runs/full/20260927T215941Z/ y YOLO26n runs/full/20260928T003017Z/, con sus evaluaciones en mission_eval/
Evaluación por misión	mission_eval/val.json y val.md de cada ejecución; código en tp4/mission.py
Evaluación de resolución	mission_eval/resolution_val.json; código en tp4/inference.py
Evaluación en video	mission_eval/video_val.json y caché de predicciones en mission_eval/video_cache/; código en tp4/video.py
VisDrone-VID validación	data/raw/VisDrone2019-VID-val/; SHA-256 del ZIP en data/archives/VisDrone2019-VID-val.json
Análisis comparativo	analysis/comparacion_yolo11_yolo26.md y figuras
Auditoría de datos	data/audit.json, data/box_review.png
Paquete OBC vigente	benchmark_artifacts/package_20260928T020732595419Z/ (YOLO26n reentrenado, persona y vehículo, 1280 px)
Guía operativa OBC	BENCHMARK_OBC.md
Código del benchmark	tp4/benchmark.py, bench_common.py, bench_worker.py, bench_report.py
Wiki y generador del informe	wiki/, tp4/wiki.py, tp4/wiki_latex.py, tp4/templates/informe_unlam.tex
Informe v1 (histórico)	informe/informe_tp4-obc.pdf

Tabla 33. Ubicación de la evidencia citada. runs/, data/ y benchmark_artifacts/ no se suben a git por tamaño, salvo los resultados que lee el notebook (JSON, figuras y el video de la demo), que sí se versionan para que funcione en Colab.

La ejecución 20260925T012307Z figura como *running* pero solo tiene seis épocas registradas; se excluyó por incompleta. El artefacto epoch_051 de YOLO26n proviene de una llamada adicional al finalizar el entrenamiento y no corresponde a una época entrenada.